

Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business[Go from Data to Strategy: Tepper School of Business](#)

Develop Your First Neural Network with PyTorch, Step by Step

by [Adrian Tam](#) on [April 8, 2023](#) in [Deep Learning with PyTorch](#)  13

Share

Share

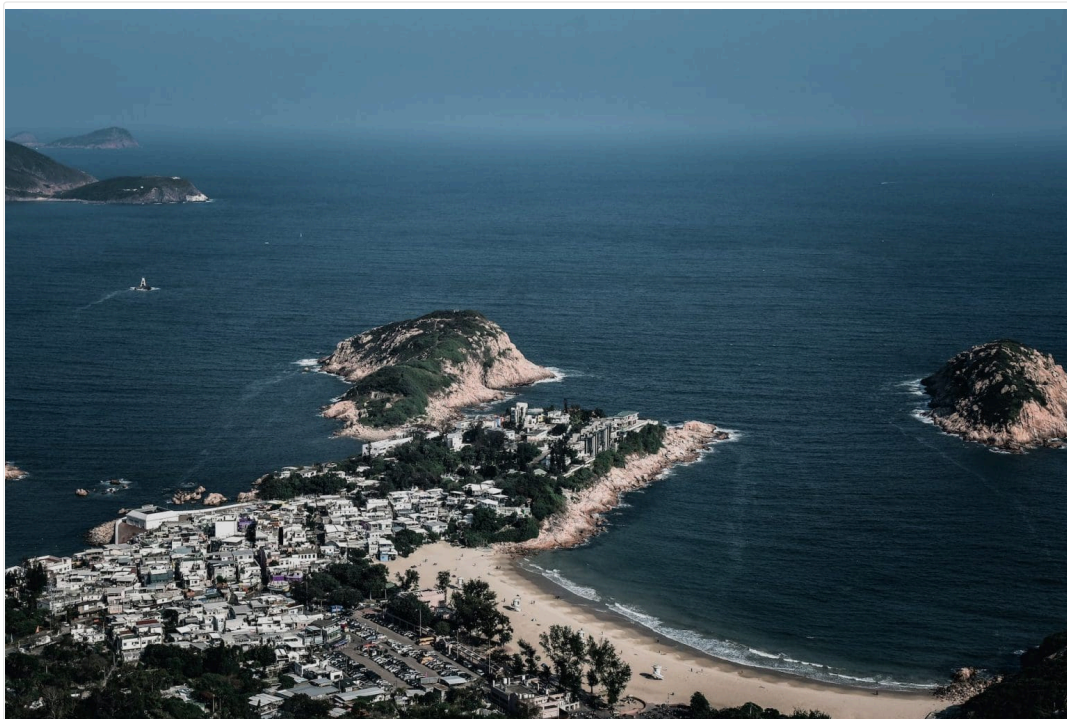
Post

PyTorch is a powerful Python library for building deep learning models. It provides everything you need to define and train a neural network and use it for inference. You don't need to write much code to complete all this. In this post, you will discover how to create your first deep learning neural network model in Python using PyTorch. After completing this post, you will know:

- How to load a CSV dataset and prepare it for use with PyTorch
- How to define a Multilayer Perceptron model in PyTorch
- How to train and evaluate a PyTorch model on a validation dataset

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Develop your First neural network with PyTorch, step by step
Photo by [drown_in_city](#). Some rights reserved.

Overview

There is not a lot of code required. You will go over it slowly so that you will know how to create your own models in the future. The steps you will learn in this post are as follows:

- Load Data
- Define PyTorch Model
- Define Loss Function and Optimizers
- Run a Training Loop

- Evaluate the Model
- Make Predictions

Load Data

The first step is to define the functions and classes you intend to use in this post. You will use the NumPy library to load your dataset and the PyTorch library for deep learning models.

The imports required are listed below:

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
```

You can now load your dataset.

In this post, you will use the Pima Indians onset of diabetes dataset. This has been a standard machine learning dataset since the early days of the field. It describes patient medical record data for Pima Indians and whether they had an onset of diabetes within five years.

It is a binary classification problem (onset of diabetes as 1 or not as 0). All the input variables that describe each patient are transformed and numerical. This makes it easy to use directly with neural networks that expect numerical input and output values and is an ideal choice for our first neural network in PyTorch.

You can also download it [here](#).

Download the dataset and place it in your local working directory, the same location as your Python file. Save it with the filename `pima-indians-diabetes.csv`. Take a look inside the file; you should see rows of data like the following:

```
1 6,148,72,35,0,33.6,0.627,50,1
2 1,85,66,29,0,26.6,0.351,31,0
3 8,183,64,0,0,23.3,0.672,32,1
4 1,89,66,23,94,28.1,0.167,21,0
5 0,137,40,35,168,43.1,2.288,33,1
6 ...
```

You can now load the file as a matrix of numbers using the NumPy function `loadtxt()`. There are eight input variables and one output variable (the last column). You will be learning a model to map rows of input variables (X) to an output variable (y), which is often summarized as $y = f(X)$. The variables are summarized as follows:

Input Variables (X):

1. Number of times pregnant
2. Plasma glucose concentration at 2 hours in an oral glucose tolerance test
3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)
5. 2-hour serum insulin ($\mu\text{U/ml}$)
6. Body mass index (weight in kg/(height in m)²)
7. Diabetes pedigree function
8. Age (years)

Output Variables (y):

- Class label (0 or 1)

Once the CSV file is loaded into memory, you can split the columns of data into input and output variables.

The data will be stored in a 2D array where the first dimension is rows and the second dimension is columns, e.g., (rows, columns). You can split the array into two arrays by selecting subsets of columns using the standard NumPy slice operator `:`. You can select the first eight columns from index 0 to index 7 via the slice `0:8`. You can then select the output column (the 9th variable) via index 8.

```
1 ...
2
3 # load the dataset, split into input (X) and output (y) variables
4 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
5 X = dataset[:,0:8]
6 y = dataset[:,8]
```

But these data should be converted to PyTorch tensors first. One reason is that PyTorch usually operates in a 32-bit floating point while NumPy, by default, uses a 64-bit floating point. Mix-and-match is not allowed in most operations. Converting to PyTorch tensors can avoid the implicit conversion that may cause problems. You can also take this chance to correct the shape to fit what PyTorch would expect, e.g., prefer $n \times 1$ matrix over n -vectors.

To convert, create a tensor out of NumPy arrays:

```
1 X = torch.tensor(X, dtype=torch.float32)
2 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
```

You are now ready to define your neural network model.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Define the Model

Indeed, there are two ways to define a model in PyTorch. The goal is to make it like a function that takes an input and returns an output.

A model can be defined as a sequence of layers. You create a `Sequential` model with the layers listed out. The first thing you need to do to get this right is to ensure the first layer has the correct number of input features. In this example, you can specify the input dimension `8` for the eight input variables as one vector.

The other parameters for a layer or how many layers you need for a model is not an easy question. You may use heuristics to help you design the model, or you can refer to other people's designs in dealing with a similar problem. Often, the best neural network structure is found through a process of trial-and-error experimentation. Generally, you need a network large enough to capture the structure of the problem but small enough to make it fast. In this example, let's use a fully-connected network structure with three layers.

Fully connected layers or dense layers are defined using the `Linear` class in PyTorch. It simply means an operation similar to matrix multiplication. You can specify the number of inputs as the first argument and the number of outputs as the second argument. The number of outputs is sometimes called the number of neurons or number of nodes in the layer.

You also need an activation function **after** the layer. If not provided, you just take the output of the matrix multiplication to the next step, or sometimes you call it using linear activation, hence the name of the layer.

In this example, you will use the rectified linear unit activation function, referred to as ReLU, on the first two layers and the sigmoid function in the output layer.

A sigmoid on the output layer ensures the output is between 0 and 1, which is easy to map to either a probability of class 1 or snap to a hard classification of either class by a cut-off threshold of 0.5. In the past, you might have used sigmoid and tanh activation functions for all layers, but it turns out that sigmoid activation can lead to the problem of vanishing gradient in deep neural networks, and ReLU activation is found to provide better performance in terms of both speed and accuracy.

You can piece it all together by adding each layer such that:

- The model expects rows of data with 8 variables (the first argument at the first layer set to 8)
- The first hidden layer has 12 neurons, followed by a ReLU activation function
- The second hidden layer has 8 neurons, followed by another ReLU activation function
- The output layer has one neuron, followed by a sigmoid activation function

```
1 ...
2
3 model = nn.Sequential(
4     nn.Linear(8, 12),
5     nn.ReLU(),
6     nn.Linear(12, 8),
7     nn.ReLU(),
8     nn.Linear(8, 1),
9     nn.Sigmoid())
```

You can check the model by printing it out as follows:

```
1 print(model)
```

You will see:

```

1 Sequential(
2   (0): Linear(in_features=8, out_features=12, bias=True)
3   (1): ReLU()
4   (2): Linear(in_features=12, out_features=8, bias=True)
5   (3): ReLU()
6   (4): Linear(in_features=8, out_features=1, bias=True)
7   (5): Sigmoid()
8 )

```

You are free to change the design and see if you get a better or worse result than the subsequent part of this post.

But note that, in PyTorch, there is a more verbose way of creating a model. The model above can be created as a Python class inherited from the `nn.Module`:

```

1 ...
2
3 class PimaClassifier(nn.Module):
4     def __init__(self):
5         super().__init__()
6         self.hidden1 = nn.Linear(8, 12)
7         self.act1 = nn.ReLU()
8         self.hidden2 = nn.Linear(12, 8)
9         self.act2 = nn.ReLU()
10        self.output = nn.Linear(8, 1)
11        self.act_output = nn.Sigmoid()
12
13    def forward(self, x):
14        x = self.act1(self.hidden1(x))
15        x = self.act2(self.hidden2(x))
16        x = self.act_output(self.output(x))
17        return x
18
19 model = PimaClassifier()
20 print(model)

```

In this case, the model printed will be:

```

1 PimaClassifier(
2   (hidden1): Linear(in_features=8, out_features=12, bias=True)
3   (act1): ReLU()
4   (hidden2): Linear(in_features=12, out_features=8, bias=True)
5   (act2): ReLU()
6   (output): Linear(in_features=8, out_features=1, bias=True)
7   (act_output): Sigmoid()
8 )

```

In this approach, a class needs to have all the layers defined in the constructor because you need to prepare all its components when it is created, but the input is not yet provided. Note that you also need to call the parent class's constructor (the line `super().__init__()`) to bootstrap your model. You also need to define a `forward()` function in the class to tell, if an input tensor `x` is provided, how you produce the output tensor in return.

You can see from the output above that the model remembers how you call each layer.

Preparation for Training

A defined model is ready for training, but you need to specify what the goal of the training is. In this example, the data has the input features X and the output label y . You want the neural network model to produce an output that is as close to y as possible. Training a network means finding the best set of weights to map inputs to outputs in your dataset. The loss function is the metric to measure the prediction's distance to y . In this example, you should use binary cross entropy because it is a binary classification problem.

Once you decide on the loss function, you also need an optimizer. The optimizer is the algorithm you use to adjust the model weights progressively to produce a better output. There are many optimizers to choose from, and in this example, Adam is used. This popular version of gradient descent can automatically tune itself and gives good results in a wide range of problems.

```

1 loss_fn = nn.BCELoss() # binary cross entropy
2 optimizer = optim.Adam(model.parameters(), lr=0.001)

```

The optimizer usually has some configuration parameters. Most notably, the learning rate `lr`. But all optimizers need to know what to optimize. Therefore, you pass on `model.parameters()`, which is a generator of all parameters from the model you created.

Training a Model

You have defined your model, the loss metric, and the optimizer. It is ready for training by executing the model on some data.

Training a neural network model usually takes in epochs and batches. They are idioms for how data is passed to a model:

- **Epoch**: Passes the entire training dataset to the model once
- **Batch**: One or more samples passed to the model, from which the gradient descent algorithm will be executed for one iteration

Simply speaking, the entire dataset is split into batches, and you pass the batches one by one into a model using a training loop. Once you have exhausted all the batches, you have finished one epoch. Then you can start over again with the same dataset and start the second epoch, continuing to refine the model. This process repeats until you are satisfied with the model's output.

The size of a batch is limited by the system's memory. Also, the number of computations required is linearly proportional to the size of a batch. The total number of batches over many epochs is how many times you run the gradient descent to refine the model. It is a trade-off that you want more iterations for the gradient descent so you can produce a better model, but at the same time, you do not want the training to take too long to complete. The number of epochs and the size of a batch can be chosen experimentally by trial and error.

The goal of training a model is to ensure it learns a good enough mapping of input data to output classification. It will not be perfect, and errors are inevitable. Usually, you will see the amount of error reducing when in the later epochs, but it will eventually level out. This is called model convergence.

The simplest way to build a training loop is to use two nested for-loops, one for epochs and one for batches:

```
1 n_epochs = 100
2 batch_size = 10
3
4 for epoch in range(n_epochs):
5     for i in range(0, len(X), batch_size):
6         Xbatch = X[i:i+batch_size]
7         y_pred = model(Xbatch)
8         ybatch = y[i:i+batch_size]
9         loss = loss_fn(y_pred, ybatch)
10        optimizer.zero_grad()
11        loss.backward()
12        optimizer.step()
13    print(f'Finished epoch {epoch}, latest loss {loss}')
```

When this runs, it will print the following:

```
1 Finished epoch 0, latest loss 0.6271069645881653
2 Finished epoch 1, latest loss 0.6056771874427795
3 Finished epoch 2, latest loss 0.5916517972946167
4 Finished epoch 3, latest loss 0.5822567939758301
5 Finished epoch 4, latest loss 0.5682642459869385
6 Finished epoch 5, latest loss 0.5640913248062134
7 ...
```

Evaluate the Model

You have trained our neural network on the entire dataset, and you can evaluate the performance of the network on the same dataset. This will only give you an idea of how well you have modeled the dataset (e.g., train accuracy) but no idea of how well the algorithm might perform on new data. This was done for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.

You can evaluate your model on your training dataset in the same way you invoked the model in training. This will generate predictions for each input, but then you still need to compute a score for the evaluation. This score can be the same as your loss function or something different. Because you are doing binary classification, you can use accuracy as your evaluation score by converting the output (a floating point in the range of 0 to 1) to an integer (0 or 1) and compare to the label we know.

This is done as follows:

```
1 # compute accuracy (no_grad is optional)
2 with torch.no_grad():
3     y_pred = model(X)
4
5 accuracy = (y_pred.round() == y).float().mean()
6 print(f"Accuracy {accuracy}")
```

The `round()` function rounds off the floating point to the nearest integer. The `==` operator compares and returns a Boolean tensor, which can be converted to floating point numbers 1.0 and 0.0. The `mean()` function will provide you the count of the number of 1's (i.e., prediction matches the label) divided by the total number of samples. The `no_grad()` context is optional but suggested, so you relieve `y_pred` from remembering how it comes up with the number since you are not going to do differentiation on it.

Putting everything together, the following is the complete code.

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5
6 # load the dataset, split into input (X) and output (y) variables
7 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
8 X = dataset[:,0:8]
9 y = dataset[:,8]
10
11 X = torch.tensor(X, dtype=torch.float32)
12 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
13
14 # define the model
```

```

15 model = nn.Sequential(
16     nn.Linear(8, 12),
17     nn.ReLU(),
18     nn.Linear(12, 8),
19     nn.ReLU(),
20     nn.Linear(8, 1),
21     nn.Sigmoid()
22 )
23 print(model)
24
25 # train the model
26 loss_fn = nn.BCELoss() # binary cross entropy
27 optimizer = optim.Adam(model.parameters(), lr=0.001)
28
29 n_epochs = 100
30 batch_size = 10
31
32 for epoch in range(n_epochs):
33     for i in range(0, len(X), batch_size):
34         Xbatch = X[i:i+batch_size]
35         y_pred = model(Xbatch)
36         ybatch = y[i:i+batch_size]
37         loss = loss_fn(y_pred, ybatch)
38         optimizer.zero_grad()
39         loss.backward()
40         optimizer.step()
41     print(f'Finished epoch {epoch}, latest loss {loss}')
42
43 # compute accuracy (no_grad is optional)
44 with torch.no_grad():
45     y_pred = model(X)
46     accuracy = (y_pred.round() == y).float().mean()
47     print(f'Accuracy {accuracy}')

```

You can copy all the code into your Python file and save it as "pytorch_network.py" in the same directory as your data file "pima-indians-diabetes.csv". You can then run the Python file as a script from your command line.

Running this example, you should see that the training loop progresses on each epoch with the loss with the final accuracy printed last. Ideally, you would like the loss to go to zero and the accuracy to go to 1.0 (e.g., 100%). This is not possible for any but the most trivial machine learning problems. Instead, you will always have some error in your model. The goal is to choose a model configuration and training configuration that achieves the lowest loss and highest accuracy possible for a given dataset.

Neural networks are stochastic algorithms, meaning that the same algorithm on the same data can train a different model with different skill each time the code is run. This is a feature, not a bug. The variance in the performance of the model means that to get a reasonable approximation of how well your model is performing, you may need to fit it many times and calculate the average of the accuracy scores. For example, below are the accuracy scores from re-running the example five times:

```

1 Accuracy: 0.7604166865348816
2 Accuracy: 0.7838541865348816
3 Accuracy: 0.7669270634651184
4 Accuracy: 0.7721354365348816
5 Accuracy: 0.7669270634651184

```

You can see that all accuracy scores are around 77%, roughly.

Make Predictions

You can adapt the above example and use it to generate predictions on the training dataset, pretending it is a new dataset you have not seen before. Making predictions is as easy as calling the model as if it is a function. You are using a sigmoid activation function on the output layer so that the predictions will be a probability in the range between 0 and 1. You can easily convert them into a crisp binary prediction for this classification task by rounding them. For example:

```

1 ...
2
3 # make probability predictions with the model
4 predictions = model(X)
5 # round predictions
6 rounded = predictions.round()

```

Alternately, you can convert the probability into 0 or 1 to predict crisp classes directly; for example:

```

1 ...
2 # make class predictions with the model
3 predictions = (model(X) > 0.5).int()

```

The complete example below makes predictions for each example in the dataset, then prints the input data, predicted class, and expected class for the first five examples in the dataset.

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5
6 # load the dataset, split into input (X) and output (y) variables

```



```

7 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
8 X = dataset[:,0:8]
9 y = dataset[:,8]
10
11 X = torch.tensor(X, dtype=torch.float32)
12 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
13
14 # define the model
15 class PimaClassifier(nn.Module):
16     def __init__(self):
17         super().__init__()
18         self.hidden1 = nn.Linear(8, 12)
19         self.act1 = nn.ReLU()
20         self.hidden2 = nn.Linear(12, 8)
21         self.act2 = nn.ReLU()
22         self.output = nn.Linear(8, 1)
23         self.act_output = nn.Sigmoid()
24
25     def forward(self, x):
26         x = self.act1(self.hidden1(x))
27         x = self.act2(self.hidden2(x))
28         x = self.act_output(self.output(x))
29         return x
30
31 model = PimaClassifier()
32 print(model)
33
34 # train the model
35 loss_fn = nn.BCELoss() # binary cross entropy
36 optimizer = optim.Adam(model.parameters(), lr=0.001)
37
38 n_epochs = 100
39 batch_size = 10
40
41 for epoch in range(n_epochs):
42     for i in range(0, len(X), batch_size):
43         Xbatch = X[i:i+batch_size]
44         y_pred = model(Xbatch)
45         ybatch = y[i:i+batch_size]
46         loss = loss_fn(y_pred, ybatch)
47         optimizer.zero_grad()
48         loss.backward()
49         optimizer.step()
50
51 # compute accuracy
52 y_pred = model(X)
53 accuracy = (y_pred.round() == y).float().mean()
54 print(f"Accuracy {accuracy}")
55
56 # make class predictions with the model
57 predictions = (model(X) > 0.5).int()
58 for i in range(5):
59     print('%s => %d (expected %d)' % (X[i].tolist(), predictions[i], y[i]))

```

This code uses a different way of building the model but should functionally be the same as before. After the model is trained, predictions are made for all examples in the dataset, and the input rows and predicted class value for the first five examples are printed and compared to the expected class value. You can see that most rows are correctly predicted. In fact, you can expect about 77% of the rows to be correctly predicted based on your estimated performance of the model in the previous section.

```

1 [6.0, 148.0, 72.0, 35.0, 0.0, 33.599998474121094, 0.6269999742507935, 50.0] => 1 (expected 1)
2 [1.0, 85.0, 66.0, 29.0, 0.0, 26.600000381469727, 0.35100001096725464, 31.0] => 0 (expected 0)
3 [8.0, 183.0, 64.0, 0.0, 0.0, 23.299999237060547, 0.671999990940094, 32.0] => 1 (expected 1)
4 [1.0, 89.0, 66.0, 23.0, 94.0, 28.100000381469727, 0.16699999570846558, 21.0] => 0 (expected 0)
5 [0.0, 137.0, 40.0, 35.0, 168.0, 43.099998474121094, 2.2880001068115234, 33.0] => 1 (expected 1)

```

Further Reading

To learn more about deep learning and PyTorch, take a look at some of these:

Books

- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. [Deep Learning](#). MIT Press, 2016. (Online version).

APIs

- [PyTorch documentation](#)

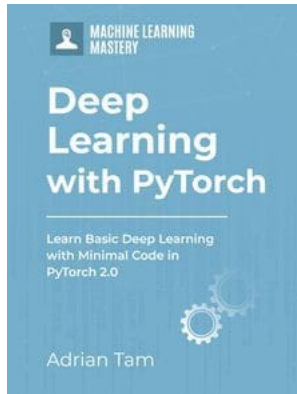
Summary

In this post, you discovered how to create your first neural network model using PyTorch. Specifically, you learned the key steps in using PyTorch to create a neural network or deep learning model step by step, including:

- How to load data
- How to define a neural network in PyTorch

- How to train a model on data
- How to evaluate a model
- How to make predictions with the model

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

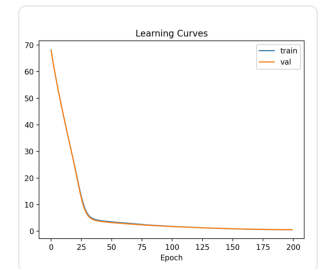
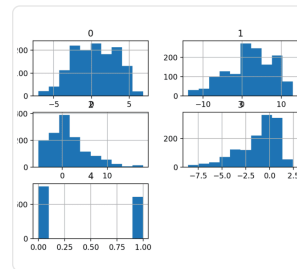
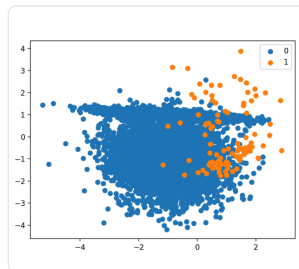
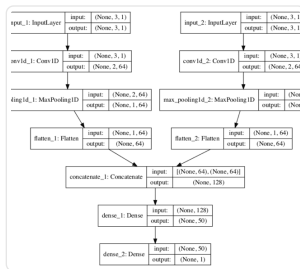
SEE WHAT'S INSIDE

Share

Share

Post

More On This Topic

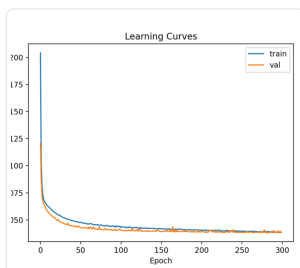


How to Develop Convolutional Neural Network Models...

How to Develop a Cost-Sensitive Neural Network for...

Develop a Neural Network for Banknote Authentication

Develop a Neural Network for Cancer Survival Dataset



Develop a Neural Network for Woods Mammography Dataset

Encoder-Decoder Recurrent Neural Network Models for...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)[◀ Building Multilayer Perceptron Models in PyTorch](#)[Creating a Training Loop for PyTorch Models ▶](#)

13 Responses to *Develop Your First Neural Network with PyTorch, Step by Step*



Sneha Ramachandran October 12, 2023 at 9:31 am #

REPLY ↩

Wonderful article! I love all of the little explanations that leave no room for confusion. Thank you!!!



James Carmichael October 13, 2023 at 9:54 am #

REPLY ↩

Thank you Sneha for your support and feedback!



Alberto November 1, 2023 at 3:02 am #

REPLY ↩

Beautiful article!

Basic question: when training in batches, is the loss computed for the entire batch, and is the gradient step taken with respect to the loss of the entire batch rather than any individual data point? If that's the case, do I also need to aggregate the various losses computed in this step?

Thanks!



Adrian Tam November 1, 2023 at 5:33 am #

REPLY ↩

Yes, entire batch. Each batch is one update to the model, but the update is based on some average metric over all individual data point in the batch. Therefore, a large batch give you better average, but the training should be more influenced by the total number of batches run.



Ani November 13, 2023 at 10:04 pm #

REPLY ↩

Nice article! How did you calculate the size of input to the first hidden layer. The explanations says - "The first hidden layer has 12 neurons" . How is this "12" calculated ?



James Carmichael November 14, 2023 at 10:29 am #

REPLY ↩

Hi Ani...This number is a hyperparameter that can be adjusted and optimized.



Milan May 12, 2024 at 5:40 pm #

REPLY ↩

Spelling correction inside the Hash Tag: "In this #pose#, you will discover how to create your first deep learning neural network model in Python using PyTorch. After completing this post, you will know:"



James Carmichael May 13, 2024 at 7:41 am #

REPLY ↩

Hi Milan...We appreciate your support and feedback!



Arun July 21, 2024 at 5:49 am #

REPLY ↩

(")" was missing in first code snippet under 'Define the Model'.. very nice article especially for beginners



James Carmichael July 21, 2024 at 6:19 am #

REPLY ↩

Thank you for your feedback Arun!



Abel November 6, 2024 at 2:21 pm #

REPLY ↩

When dividing the training dataset into batches, why don't we randomize the division and keep using the same division for all epochs? Wouldn't that cause a generalization issue? Or PyTorch training implicitly handles it.



James Carmichael November 7, 2024 at 10:10 am #

REPLY ↩

Hi Abel...Great question! When training models in PyTorch (or similar frameworks), we typically divide the data into batches using **"data loaders"**, which shuffle the data at the beginning of each epoch. Here's how it works and why it matters:

1. **"Shuffling to Avoid Generalization Issues"**: Shuffling the dataset at each epoch helps ensure that the model doesn't learn in a fixed pattern from batch division, which could indeed lead to overfitting or generalization issues. By re-randomizing the batches each epoch, the model gets exposed to a diverse range of data samples in each batch, leading to a more generalized learning process.
2. **"PyTorch's DataLoader and Shuffling"**: PyTorch's `DataLoader` has a `shuffle=True` parameter, which, when set, will shuffle the data at the start of each epoch. This randomized batching is beneficial, as it prevents the model from seeing data in the same order every time, helping it learn more robust patterns.
3. **"Why We Don't Keep Fixed Batches Across Epochs"**: If we kept the same fixed batch division across epochs, the model could pick up on patterns specific to each batch and fail to generalize. This would be like showing the model the same playlist of data repeatedly, rather than shuffling the playlist to ensure it's exposed to variations each time.

Example Code in PyTorch

```
python
from torch.utils.data import DataLoader, TensorDataset

# Assuming train_data and train_labels are your dataset and labels
train_dataset = TensorDataset(train_data, train_labels)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True) # shuffle=True for random batches

# In each epoch, the DataLoader will shuffle the dataset again.
for epoch in range(num_epochs):
    for batch in train_loader:
        # training loop here
```

In summary, shuffling batches each epoch is crucial for generalization, and PyTorch's `DataLoader` helps handle this automatically. So, as long as you're using `shuffle=True` with your `DataLoader`, you're covered!



paul February 2, 2026 at 1:28 pm #

REPLY ↩

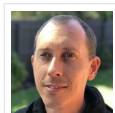
merci encore c'était très instructif

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

**Welcome!**I'm *Jason Brownlee* PhDand I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:****Picked for you:**[PyTorch Tutorial: How to Develop Deep Learning Models with Python](#)[LSTM for Time Series Prediction in PyTorch](#)[Deep Learning with PyTorch \(9-Day Mini-Course\)](#)[Calculating Derivatives in PyTorch](#)[Building a Regression Model in PyTorch](#)**Loving the Tutorials?**

The [Deep Learning with PyTorch](#) EBook
is where you'll find the **Really Good** stuff.

[>> SEE WHAT'S INSIDE](#)

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.

